

AI Research Assistant

Planejamento do Projeto

1. Contexto e Motivação

Quem sou eu

Sou estudante de Engenharia da Computação no Insper, atualmente em Chicago no SISRM da University of Chicago. Em 2026–27 serei Visiting Undergraduate Student em Harvard. Me interesso por tecnologia, IA, empreendedorismo e impacto social, e quero me tornar uma líder de tecnologia com profundidade técnica real para construir produtos baseados em IA.

Percebi que meu maior desafio não é falta de oportunidades ou conteúdo, mas falta de direção. Decidi parar de consumir conteúdo passivamente e aprender construindo um projeto real.

Motivação

O principal desafio identificado não é falta de oportunidades ou conteúdo, mas falta de direção. Em vez de consumir conteúdo passivamente, a decisão foi focar em um projeto pessoal que permita **aprender construindo**.

- Ensinar conceitos modernos de IA na prática
- Expor a NLP, LLMs, RAG, embeddings e retrieval
- Gerar algo útil no dia a dia
- Ser um portfólio técnico relevante
- Poder evoluir ao longo do tempo
- Ajudar a ler e entender papers acadêmicos

2. Problema e Visão do Produto

Problema

Acesso a uma enorme quantidade de papers via University of Chicago, mas eles são longos, técnicos e difíceis de navegar rapidamente. A necessidade é de uma ferramenta que auxilie na compreensão, extração de informações e exploração da literatura científica.

Funcionalidades planejadas

- Upload de PDF
- Resumo automático
- Extração estruturada (metodologia, datasets, conclusões, limitações)
- Identificação de datasets utilizados
- Geração automática de perguntas e análise crítica
- Busca semântica de papers relacionados
- Chat com os papers (RAG)
- Base pessoal de conhecimento acadêmico

3. Arquitetura e Stack Tecnológica

Decisões de arquitetura

Banco de dados	PostgreSQL	Padrão do mercado e de startups. ORM: SQLAlchemy + Alembic para migrações. Local via Docker, produção via Railway ou Supabase.
Backend	FastAPI (Python)	API REST moderna, tipagem nativa com Pydantic, async nativo.
Frontend	React	Single-page application com componentes reutilizáveis. Servirá Upload, Dashboard, PaperDetail e Chat.
LLM	Anthropic (Claude) ou OpenAI	Iniciar com modelos menores (Haiku / GPT-4o-mini) para controle de custo. Escalar conforme necessidade.
Embeddings	text-embedding-3-small	Modelo da OpenAI, custo baixíssimo, qualidade suficiente para busca semântica acadêmica.
Vector DB	ChromaDB (local) → Pinecone (cloud)	ChromaDB para desenvolvimento; Pinecone quando precisar de escala ou persistência em produção.
Monorepo	Uma única repo com /backend e /frontend	Facilita versionamento conjunto, CI/CD unificado e documentação centralizada.

Deploy e Custo Estimado

Componente	Plataforma	Custo estimado
Backend (FastAPI)	Railway	~\$0–5/mês (free tier generoso)
Frontend (React)	Vercel	Gratuito para projetos pessoais
Banco de dados (Postgres)	Railway ou Supabase	Gratuito no free tier
LLM API	Anthropic / OpenAI	~\$3–8/mês com uso moderado
Embeddings	OpenAI (text-embedding-3-small)	Centavos por token
Vector DB	ChromaDB local → Pinecone	\$0 local; ~\$0–10/mês cloud
Total estimado		~\$5–15/mês em uso ativo

4. Roadmap das Sprints

Sprint 0 PDF → Resumo Simples **Semana 1**

MVP funcional. Você já conhece os ingredientes, só precisa montá-los.

Conceitos aprendidos

• LLMs (prompting básico) • OpenAI/Anthropic API • PDF parsing • FastAPI • React (upload + exibição)

O que construir

- Ler PDF com PyMuPDF, extrair texto bruto
- Enviar texto para API (GPT-4o-mini ou Claude Haiku)
- Prompt de resumo: objetivo, método, conclusões
- API endpoint POST /papers/upload
- Frontend: tela de upload + exibição do resumo
- Persistir paper e resumo no PostgreSQL

Stack

`Python 3.11+` · `FastAPI` · `PyMuPDF` · `Anthropic SDK` · `React` · `PostgreSQL` · `Docker`

Sprint 1 Extração Estruturada **Semana 2**

Saída em JSON estruturado ao invés de texto livre. Um salto enorme de utilidade.

Conceitos aprendidos

• Structured Outputs / JSON mode • Pydantic models • Prompt Engineering avançado • Few-shot examples

O que construir

- Definir schema Pydantic: objetivo, metodologia, dados, conclusões, limitações
- Usar JSON mode (OpenAI) ou tool use (Claude)
- Validar e tratar erros de parse
- Exibir campos como cards no frontend
- Salvar estrutura no PostgreSQL (coluna JSONB)

Stack

`Pydantic v2` · `OpenAI JSON mode` / `Claude tool use` · `PostgreSQL JSONB` · `React components`

Sprint 2 Perguntas e Análise Crítica **Semanas 3–4**

O assistente começa a pensar junto com você, não só resumir.

Conceitos aprendidos

• Chain of Thought prompting • Persona prompting • Geração de perguntas • Crítica estruturada

O que construir

- Gerar perguntas de leitura crítica (metodologia, vieses, lacunas)
- Criar modo 'peer reviewer': o modelo critica o paper
- Gerar perguntas de pesquisa futura a partir das limitações
- Salvar histórico de perguntas por paper no banco
- Comparação entre dois papers (prompt multi-documento)

Stack

`Prompts multi-turn` · `PostgreSQL` · `React tabs` e `comparação side-by-side`

Sprint 3

Busca Semântica por Embeddings Semanas 5–7

O salto mais importante do projeto. Você vai entender IA de verdade aqui.

Conceitos aprendidos

• Embeddings • Vector databases • Semantic search • Cosine similarity • Chunking strategies

O que construir

- Gerar embeddings dos papers com text-embedding-3-small
- Indexar em ChromaDB (local) ou Pinecone (cloud)
- Busca: digitar conceito → encontrar papers relacionados
- Visualizar similaridade entre papers (heatmap)
- Recomendar papers similares ao que está sendo lido

Stack

`text-embedding-3-small` · `ChromaDB` · `numpy` / `scipy` · `React search UI`

Sprint 4

Chat com o Paper (RAG) Semanas 8–10

O produto completo. Você implementa RAG do zero — a arquitetura mais usada em produção.

Conceitos aprendidos

• RAG (Retrieval-Augmented Generation) • Chunking + overlapping • Vector retrieval • Context injection • Conversation memory

O que construir

- Dividir papers em chunks com overlap (LangChain TextSplitter)
- Indexar chunks por paper no ChromaDB
- Query → retrieve top-k chunks → injetar no prompt
- Conversa multi-turn com histórico no contexto
- Mostrar na UI quais trechos foram usados (sources)
- Avaliar qualidade: respostas com vs sem RAG

Stack

`LangChain` (opcional) · `ChromaDB` · `FastAPI` `WebSocket` ou `SSE` · `React chat interface`

5. Estratégia de Documentação

Todo o conhecimento gerado no projeto fica em **docs/** como arquivos .md, versionados junto ao código no GitHub.

Arquivo	Conteúdo
<code>docs/sprints/sprint-N.md</code>	Plano da sprint, decisões tomadas, o que funcionou e o que não funcionou, aprendizados
<code>docs/architecture.md</code>	Diagrama de arquitetura, decisões de design e justificativas
<code>docs/api.md</code>	Contratos de API entre frontend e backend (endpoints, schemas)
<code>docs/prompts.md</code>	Versões dos prompts, iterações, resultados — o arquivo mais valioso do projeto
<code>docs/costs.md</code>	Controle de gastos: tokens por feature, custo por paper, alertas de anomalias